國立臺北科技大學
National Taipei University of Technology

資料結構
Data Structure

Lab 06

姓名: 盧敬恩
學號: 114310123

Lab06-Q1and Q2(queue)

利用queue資料結構完成21點遊戲

Code

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <random>
#include <ctime>
using namespace std;
struct Player {
    string name;
    int score;
};

class Card {//定義物件CARD
public:
    string suit;//成員:套組和等級
    string rank;
    Card() : suit(""), rank("") {}//建構子 清空SUIT 和 RANK
    Card(string s, string r) : suit(s), rank(r) {}//建構子:放入SUT 和 RANK
    void display() const {// 成員函式 DISPLAY顯示花色和等級
        cout << rank << " of " << suit << endl;
    }
};

class Deck {//定義物件DECK
private:
    Card cards[52];//成員:52個CARDS物件(CARDS)
    int index;
public:
    Deck() : index(0) {
//建構子 先把index設定為0接者把52張卡片從heart 到 spades 2-a按順序放入
        string suits[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
        string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q",
"K", "A" };
        int k = 0;
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 13; j++) {
                cards[k] = Card(suits[i], ranks[j]);
                k++;
            }
        }
    }
```

```
void shuffleDeck() {
 //成員函式suffle deck:功能是先建立隨機變數j做為和cards[i]交換的對象, 為了保證隨機每一張i互換時的
J都會隨機刷新一次
        for (int i = 51; i > 0; i--) {
            int j = rand() % (i + 1);
            swap(cards[i], cards[j]);//來自algorithm的語法 能互換兩者之值
        }
    }

    Card getCard(int i) const {//成員函式GETCARD 功能為回傳cards的第i位
        return cards[i];
    }
};
class Queue {//定義類別柱列
private:
    vector<Card> deck;//成員:vector deck 每一個單元都是Card
    int Front;
    int Rear;//三變數FRONT  REAR 以及容量
    int capacity;
public:
    Queue(int size) : Front(-1), Rear(-1), capacity(size) {}//建構子:設定FRONT 和
REAR=-1 以及容量

    bool isEmpty() const {
        return (Front == -1 || Front > Rear);//如果FRONT = -1或是DEQUENE所有內容時FRONT會
大於REAR, 此時回傳1
    }

    bool isFull() const {
        return (deck.size() >= capacity);//如果VECTOR DECK(也就是queue)>=設定容量=>回傳1
表示滿了
    }

    void enqueue(Card card) {//成員函式enqueue 如果滿了就回報queue已滿 無法放牌
        if (isFull()) {
            cout << "牌堆已滿,無法插入!" << endl;
            return;
        }
        if (Front == -1) Front = 0;//如果該queue為空front設為0
        deck.push_back(card);    //將card push至deck 尾端
        Rear++;              //rear標+1
    }
```

```
void dequeue() {//排出牌堆
        if (isEmpty()) { //如果空了就不排出
            cout << "牌堆已空!" << endl;
            return;
        }
        Front++;//排出卡牌時front++
        if (Front > Rear) {
//如果發生Front > Rear 代表全部排出了 將front 和rear設為負一, 將quene清空
            Front = -1;
            Rear = -1;
            deck.clear();
        }
    }

    Card getFront() {//因為在dequene的時候沒有說明把排出的值回傳 所以getfront先取得front
front再＋1
        if (isEmpty()) {
            cout << "牌堆為空,無法獲取最前端的牌!" << endl;
            return Card("", "");
        }
        return deck[Front];//回傳第front位
    }
};

// --- 遊戲邏輯函式 ---

void initializeDeck(Queue& cardDeck) {//初始化牌堆
    Deck d;
    cout << "初始牌堆: \n";
    for (int i = 0; i < 52; i++) {//該FUNCTION會將未排序的牌堆輸出一次
        d.getCard(i).display();
    }

    d.shuffleDeck(); //洗牌
    cout << "\n洗牌後的牌堆: \n";
    for (int i = 0; i < 52; i++) {//接著將所有洗好的牌顯示並ENQUEUE
        d.getCard(i).display();
        cardDeck.enqueue(d.getCard(i)); // 將洗好的牌放入 Queue
    }
    cout << "--------------------------\n";
}
```

```
void initializePlayer(Player* player, string name, Queue& cardDeck) {//這段是模擬抽牌 可以是玩家也可以是莊家
    player->name = name;
    player->score = 0;
    cout << player->name << " 抽到的牌: ";
    for (int i = 0; i < 1; i++) {
        Card drawnCard = cardDeck.getFront();//抽卡並DEQUEUE
        cardDeck.dequeue();

        cout << drawnCard.rank << " of " << drawnCard.suit << " ";

        if (drawnCard.rank == "A") player->score += 1;//如果RANK是1 +1分
        else if (drawnCard.rank == "J" || drawnCard.rank == "Q" ||
drawnCard.rank == "K")
            player->score += 10;//如果RANK是J Q K +10分
        else player->score += stoi(drawnCard.rank);//否則將RANK轉回INT格式加到
SCORE
    }
    cout << "\n" << player->name << " 玩家的初始分數: " << player->score <<
endl;//最後將SCORE輸出
}
```

```
void playerTurn(Player* player, Queue& cardDeck) {//輸入H和S至CHOISE變數 如果H就是抽一張 S
代表停牌

    char choice;
    while (player->score < 21 && !cardDeck.isEmpty()) {//抽牌的條件:玩家<21分且還有牌
        cout << player->name << " 您的手牌分數目前為: " << player->score << " 要抽牌嗎?(h
= 抽, s = 停) : ";
        cin >> choice;
        if (choice == 'h') {
            Card newCard = cardDeck.getFront();
            cardDeck.dequeue();

            if (newCard.rank == "A") player->score += 1;
            else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank ==
"K")
                player->score += 10;
            else player->score += stoi(newCard.rank);

            cout << player->name << " 抽到: " << newCard.rank << " of " <<
newCard.suit
                << " 總分: " << player->score << endl;

            if (player->score > 21) {//如總分>21結束遊戲
                cout << player->name << " 爆掉了!遊戲結束。\n";
                return;
            }
        }
        else if (choice == 's') {//如輸入S停牌 並結束這部分程式
            cout << player->name << " 選擇停牌,總分: " << player->score << endl;
            break;
        }
        else {//如果輸入不是H OR S, 提示輸入異常
            cout << "請輸入有效選項 (h = 抽, s = 停)!" << endl;
        }
    }
}
void dealerTurn(Player* dealer, Queue& cardDeck) {//DEALER函式和玩家函式邏輯一樣但是不能主
動選擇是否再抽
    while (dealer->score < 17 && !cardDeck.isEmpty()) {
        Card newCard = cardDeck.getFront();
        cardDeck.dequeue();
        if (newCard.rank == "A") dealer->score += 1;
        else if (newCard.rank == "J" || newCard.rank == "Q" || newCard.rank == "K")
            dealer->score += 10;
        else dealer->score += stoi(newCard.rank);
        cout << "莊家抽到: " << newCard.rank << " of " << newCard.suit
            << " 莊家目前總分: " << dealer->score << endl;
        if (dealer->score > 21) {
            cout << "莊家爆了!玩家獲勝!" << endl;
            return;
        }
    }
}
```

```
void determineWinner(Player* player, Player* dealer) {
    if (player->score > 21) { //玩家爆牌
        cout << player->name << " 爆了!莊家獲勝!\n";
    }
    else if (dealer->score > 21) {//莊家爆牌
        cout << player->name << " 贏了!\n";
    }
    else if (player->score > dealer->score) {// 兩人都沒爆,玩家分數較高
        cout << player->name << " 贏了!\n";
    }
    else if (player->score < dealer->score) {//兩人都沒爆,莊家分數較高
        cout << "莊家贏了!\n";
    }
    else {//分數一模一樣
        cout << "平手!\n";
    }
}
int main() {
    srand((unsigned int)time(0));//初始化亂樹種子
    Queue cardDeck(52);//宣告佇列cardDeck有52張牌
    initializeDeck(cardDeck);//該佇列沒有內容, 初始化即可放入沒洗過的牌
    Player player, dealer;//宣告兩個物件莊家和玩家
    cout << "\n3. 遊戲過程\n";
    initializePlayer(&player, "玩家", cardDeck);//這是第一次抽牌(玩家)
    initializePlayer(&dealer, "莊家", cardDeck);//這是第一次抽牌(莊家)
    playerTurn(&player, cardDeck);//這裡是讓玩家選擇要不要抽牌, 如果玩家在此就>21點,
會在此函式中直接判定結果
    if (player.score <= 21) {//如果玩家<=21就讓莊家抽一次
        cout << "\n 莊家回合...\n";
        dealerTurn(&dealer, cardDeck);
        if (dealer.score <= 21) {//進入該if條件是莊家抽了牌, 但是沒有爆掉
            determineWinner(&player, &dealer);//接著決定贏家 一定有結果(平手/其中一
名獲勝/其中一人更接近21)
        }
    }
    return 0;
}
```

Result

Case1:

初始牌堆1

Microsoft Visual Studio 偵錯主

```
初始牌堆:
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
```

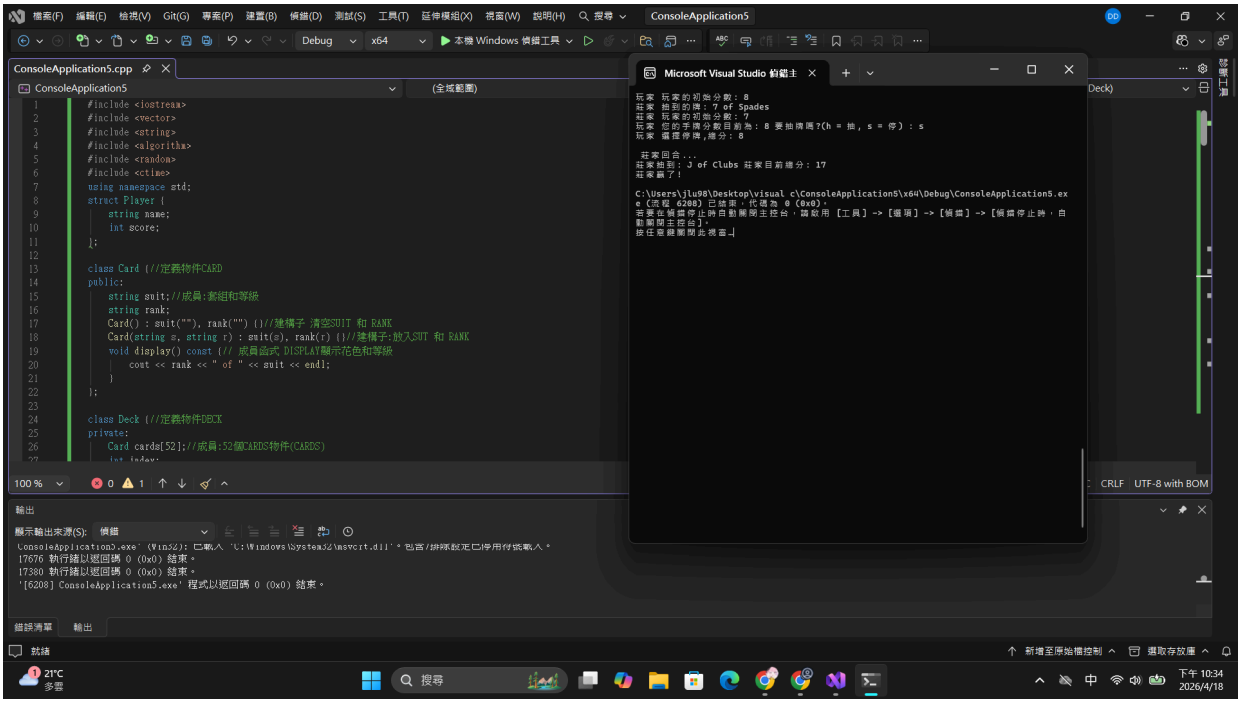
洗牌後牌堆1

```
洗牌後的牌堆：
J of Spades
9 of Spades
K of Diamonds
2 of Spades
J of Hearts
2 of Diamonds
3 of Diamonds
A of Spades
9 of Diamonds
K of Clubs
10 of Spades
3 of Clubs
3 of Spades
Q of Spades
Q of Diamonds
K of Hearts
4 of Spades
7 of Spades
3 of Hearts
8 of Hearts
8 of Spades
2 of Clubs
9 of Clubs
7 of Diamonds
6 of Diamonds
J of Diamonds
6 of Hearts
6 of Spades
2 of Hearts
5 of Spades
A of Diamonds
A of Clubs
5 of Hearts
A of Hearts
5 of Clubs
10 of Clubs
9 of Hearts
10 of Hearts
8 of Clubs
7 of Hearts
J of Clubs
Q of Hearts
K of Spades
5 of Diamonds
8 of Diamonds
4 of Diamonds
4 of Hearts
Q of Clubs
4 of Clubs
10 of Diamonds
6 of Clubs
7 of Clubs
--------------------------
```

運行結果1

```
3. 遊戲過程
玩家 抽到的牌: Q of Clubs
玩家 玩家的初始分數: 10
莊家 抽到的牌: 10 of Spades
莊家 玩家的初始分數: 10
玩家 您的手牌分數目前為: 10 要抽牌嗎?(h = 抽, s = 停) : H
請輸入有效選項 (h = 抽, s = 停)!
玩家 您的手牌分數目前為: 10 要抽牌嗎?(h = 抽, s = 停) : h
玩家 抽到: 8 of Clubs 總分: 18
玩家 您的手牌分數目前為: 18 要抽牌嗎?(h = 抽, s = 停) : h
玩家 抽到: 7 of Diamonds 總分: 25
玩家 爆掉了!遊戲結束。
```

全螢幕截圖1



Case2:
初始牌堆2

```
Microsoft Visual Studio 偵錯主

初始牌堆:
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
J of Hearts
Q of Hearts
K of Hearts
A of Hearts
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
J of Diamonds
Q of Diamonds
K of Diamonds
A of Diamonds
2 of Clubs
3 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
J of Clubs
Q of Clubs
K of Clubs
A of Clubs
```

洗牌後牌堆2

Microsoft Visual Studio 偵錯主

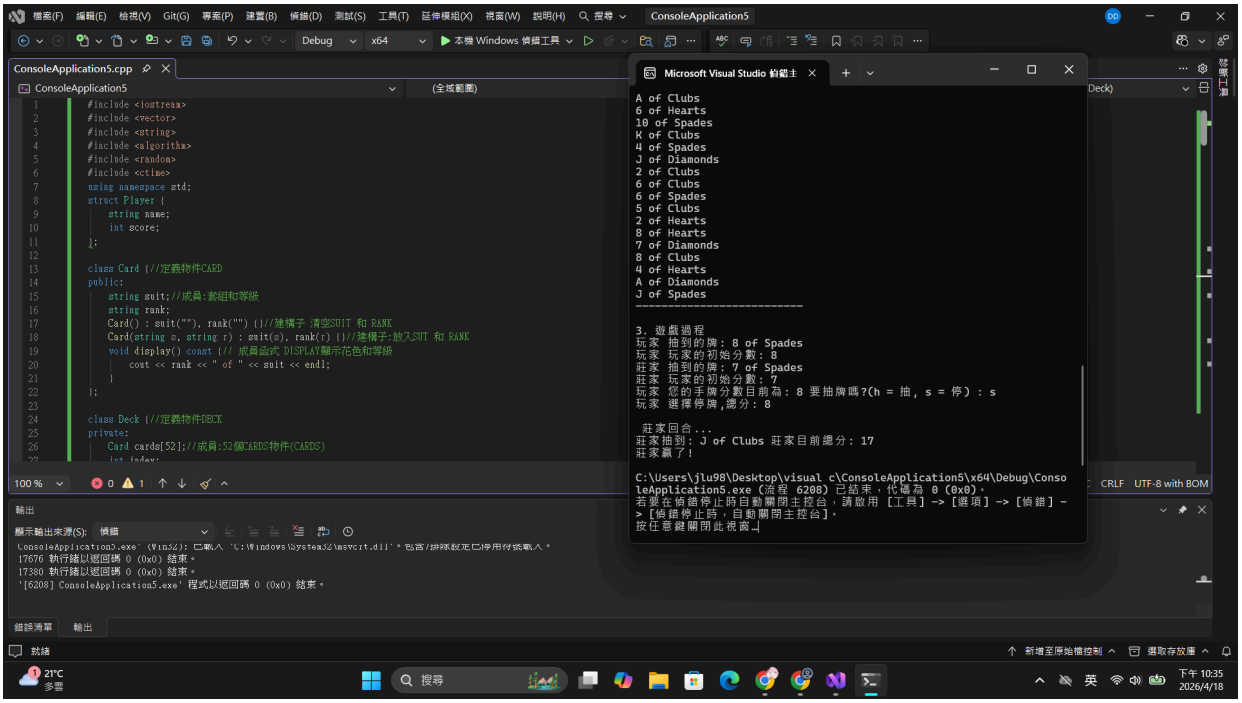
```
洗牌後的牌堆:
8 of Spades
7 of Spades
J of Clubs
3 of Clubs
4 of Clubs
4 of Diamonds
K of Diamonds
2 of Spades
9 of Diamonds
Q of Spades
Q of Clubs
A of Hearts
3 of Spades
10 of Clubs
8 of Diamonds
9 of Clubs
9 of Hearts
3 of Hearts
3 of Diamonds
9 of Spades
6 of Diamonds
Q of Hearts
A of Spades
K of Hearts
J of Hearts
10 of Hearts
K of Spades
10 of Diamonds
7 of Clubs
Q of Diamonds
7 of Hearts
2 of Diamonds
5 of Hearts
5 of Spades
5 of Diamonds
A of Clubs
6 of Hearts
10 of Spades
K of Clubs
4 of Spades
J of Diamonds
2 of Clubs
6 of Clubs
6 of Spades
5 of Clubs
2 of Hearts
8 of Hearts
7 of Diamonds
8 of Clubs
4 of Hearts
A of Diamonds
J of Spades
--------------------------
```

運行結果2

```
3. 遊戲過程
玩家 抽到的牌: 8 of Spades
玩家 玩家的初始分數: 8
莊家 抽到的牌: 7 of Spades
莊家 玩家的初始分數: 7
玩家 您的手牌分數目前為: 8 要抽牌嗎?(h = 抽, s = 停) : s
玩家 選擇停牌,總分: 8

 莊家回合...
莊家抽到: J of Clubs 莊家目前總分: 17
莊家贏了!
```

全螢幕截圖2



Discussion

完成討論:Queue與Stack在執行發牌上的差異

QUEUE佇列在程式上先將乾淨的牌放入DECK中，在DECK裡面每一張牌隨機的交換以達成洗牌
發牌時由底部那張開始一張一張的發出去，像是飲水機旁從底部(FRONT)抽取的環保紙杯，遵守FIFO
STAKE則是將牌洗好由上往下取走，像是堆在一起的盤子，或是玩撲克牌背面向上玩家輪流抽牌
此行為為FILO，在電腦中如發生中斷或是執行副程式時會把當前程式放入堆疊，先處理中斷或是副程式再回來做當時的程式。